TRA-BOT
An Intelligent Traffic Monitoring System

(By Sankeeth Kurian)

Abstract—**KEYWORDS: Traffic Safety, Image processing, AI-ML-DL, Django, Python, Raspberry Pi, R-Pi Camera**

Traffic congestion occurs due to many factors, including traffic violations, unforeseen accidents, and unbalanced traffic flow. Our proposed system detects road accidents, traffic violations, and traffic flow based on density through the use of image processing and AI-ML-DL techniques. Road accidents are one of the major concerns in traffic safety because the number of vehicles on the roads is increasing day by day, despite the use of traditional traffic management. A lot of valuable lives are being lost because of these accidents where immediate, necessary services are unavailable. Our system can detect these accidents and alert the necessary authorities in time. Here, video footage of an accident is being sent to the police and hospital authorities. For that, we have developed a web page using Django and Python programming to send the video footage to the corresponding login credentials of both the police and the hospital. The decision to be taken by them can be either accepted or rejected. Perhaps due to the inaccuracies of the trained dataset, the decision can be rejected. This is implemented as one of the intelligence parts of the system. Along with that, for the accident prediction system, we developed the intelligence part of traffic violations (over-speed detection) and traffic flow based on density detection based on the given dataset (also made as real-time using laptop and its front camera) as two separate modules that predict the occurrence of any accident. Thereby, it also provides an alert SMS to the control room of the police authority if any traffic violation or high traffic density is found, so that they can take actions or precautions to prevent the occurrence of an accident. The intelligence part of accident detection is implemented as a hardware part using a Raspberry Pi (4GB) and R-Pi camera, thereby making it real-time.

I. INTRODUCTION

In today's world, there are numerous issues concerning the common motorist and the safety of roadside construction workers who have to operate in close proximity to them. Things like negligent driving and speeding pose some serious concerns and threats to us humans. Recommendations were evaluated and examined in terms of having a device with the ability to monitor roadside construction workers as well as ongoing traffic.

According to the latest data available from the Ministry of Road Transport and Highways in India, there were a total of 4,49,002 road accidents in the country in 2019, which translates to an average of 1,229 accidents per day. These accidents resulted in 1,54,732 deaths and 4,51,361 injuries. Also, Kerala had a total of 39,986 road accidents in 2019, which means an average of around 109 accidents per day in the state. These accidents resulted in 4,267 deaths and 40,743 injuries. It's important to note that this data is from 2019, and the number of accidents may have fluctuated in subsequent years.

While traditional traffic safety methods have been effective in reducing traffic accidents and fatalities, they do have some disadvantages when compared to advanced methods. Some of these disadvantages are limited effectiveness, lack of adaptability, reliance on human behavior, and limited scope. Advanced road safety methods, on the other hand, offer several advantages over traditional methods. So, this paper deals with an advanced method of road accident prevention. New developments in traffic automation have been brought to light all around the world. From this, the idea of a device that can monitor traffic and make real-time and accurate judgements to increase safety is the main objective of this paper.

A. Purpose

The main purpose of this paper is to make use of advanced engineering technologies like AI and image processing to implement an intelligent traffic monitoring system (ITMS).

B. Objective

In the implementation of an ITMS, we give priority to the safety of people on the road. A smart accident detection and alerting system for police and hospital authorities is the main module, followed by a smart accident prevention system to prevent the occurrence of an accident, as prevention is always better than cure. For that, a traffic violation detection module (over-speed detection and alerting) and a traffic density detection module (if there is high traffic, then giving an alert) are developed as intelligent parts. The intelligent part of the accident detection part is implemented as a hardware part, thereby making it real-time.

II. LITERATURE REVIEWS

There are several techniques projected for the working of TRA-BOT. This additionally includes cameras and other hardware based solutions. Image processing and AI-ML-DL techniques is most preferred for its best optimum results.

Traffic Density Control Using Deep Learning [1] focuses on the application of deep learning techniques to manage traffic density. The paper explores the use of advanced machine learning algorithms and neural networks to analyze and control traffic flow. Accident Detection and Alert System [2] presents a novel system for detecting and alerting accidents. The paper discusses the development and implementation of an intelligent system that utilizes various technologies and sensors to identify accidents or collisions.

Examining the Role of Safety in the Low Adoption Rate of Collaborative Robots [3] investigates the factors contributing to the limited adoption of collaborative robots and specifically focuses on the role of safety considerations. Collaborative robots, also known as cobots, have the potential to work alongside humans in various industries, but their adoption has been slower than expected. Intelligent Traffic Violation Detection [4] focuses on the development and implementation of an intelligent system for detecting traffic violations. By leveraging advanced technologies and algorithms, the authors propose a novel approach to automatically identify various traffic violations, such as speeding, red light running, and improper lane changes.

Video Streaming in Web Browsers with OpenCV Flask [5] focuses on the topic of video streaming in web browsers using OpenCV and Flask. OpenCV is a popular computer vision library, and Flask is a web framework for Python. The author explores the integration of these technologies to enable real-time video streaming in web browsers. A Review of the Use of Traffic Simulation for the Evaluation of Traffic Safety Levels [6] provides a comprehensive review of the use of traffic simulation for evaluating traffic safety levels. The authors explore the potential of traffic simulation models in predicting and assessing crash occurrences. They delve into the various methodologies, techniques, and data sources employed in traffic simulation for safety analysis purposes.

Smart Accident Detection and Alert Message System [7] proposed a solution that utilizes various technologies, such as sensors, GPS, and wireless communication, to detect sudden impacts or abnormal movement patterns associated with accidents. Upon detection, the system promptly generates and sends alert messages, providing crucial information about the accident location and potentially aiding in rapid response and assistance. Traffic Signal Violation Detection Using Artificial Intelligence and Deep Learning [8] presents an innovative approach for detecting traffic signal violations using artificial intelligence (AI) and deep learning techniques. The paper proposed a system that leverages AI and deep learning algorithms to analyze real-time traffic video data and identify instances of signal violations.

Controlling Traffic with Humanoid Social Robot [9] explores the application of a humanoid social robot in traffic control. This focused on the design and implementation of a system that utilizes a humanoid social robot to assist in traffic control tasks. By leveraging advanced technologies and human-like interaction capabilities, the robot aims to enhance the efficiency and effectiveness of traffic management processes. Real-time Traffic Density and Overspeeding Detection System Using Deep Learning [10] focuses on the development of a system for real-time traffic density and overspeeding detection using deep learning techniques. They proposed an intelligent system that utilizes deep learning algorithms to analyze traffic video data and detect two important factors: traffic density and instances of overspeeding. By training the deep learning model on a large dataset of annotated video footage, the system learns to accurately identify and classify different traffic conditions and identify vehicles that are exceeding the speed limit.

HawkAI: Monitoring Road Accidents Real-time Using Deep Learning [11] presents the results of the MIDS Capstone Project conducted in the summer of 2019. The paper focuses on the development of an intelligent system called HawkAI, which utilizes deep learning techniques to monitor road accidents in real-time. The authors propose a solution that employs deep learning algorithms to analyze video data captured by road cameras and automatically detect and classify accidents. By training the deep learning model on a large dataset of annotated video footage, HawkAI can accurately identify and notify authorities about accidents as they occur. A Robotic Traffic Simulator for Teaching of Advanced Control Methods [12] discusses the development of a robotic traffic simulator aimed at teaching advanced control methods. The authors present a simulator that combines robotic vehicles and advanced control algorithms to recreate realistic traffic scenarios in an educational setting. The simulator allows students to experiment with various control methods and algorithms, enabling them to gain hands-on experience in designing and implementing control strategies for managing traffic flow.

III. Working Methodology

The TRA-BOT system developed here uses image processing and AI to detect accidents happening in real time and give an alert to a nearby hospital and police station. The model is trained with both accident and non-accident scenarios using AI (Deep Learning-CNN). The hospitals and police stations can access the alerts through a web page provided beforehand (developed using Django). Video footage from the accident scene will also be sent to both ends. Based on the footage, they can either accept or deny the alert. If accepted, services will be provided at the accident scene. This is the accident detection part of TRA-BOT. The system also detects traffic violations (like over-speeding) and traffic density (like normal or high traffic) for accident prevention. In the traffic violation part, if an over-speed is recorded by the system, it will crop the image of those vehicles and save its details to the police directory. It will send an alert SMS to the police control room (mobile phone) to take the necessary actions. Similarly, for the traffic density part, the system will classify an image as normal or high traffic. If there is high

traffic, send an alert SMS to the police control room (mobile phone) to take precautions to reduce the traffic. This form of SMS sending is implemented using the Twilio communication tool. Fig. 1 shows the basic block diagram of the proposed system.

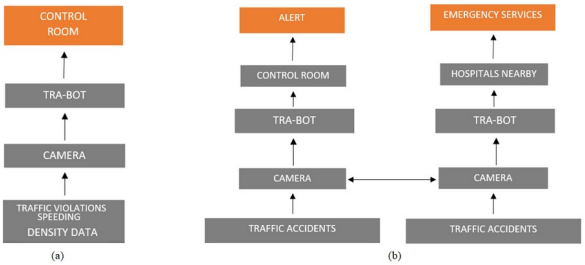

Fig.1 Basic block diagram- (a) Accident prediction system and (b) Accident detection system

A. Accident Detection Model of TRA-BOT

This model of TRA-BOT is implemented in both software and hardware. This includes training, testing, and detection of an accident (if one occurs). This works on a combination of image processing and AI, especially deep learning. It uses CNN to develop a pre-trained model called 'DetectNet' to predict whether an accident has occurred or not [2,7].

For accident detection, we made a web page using Django, a high-level web framework for building web applications written in Python. A Python script (manage.py) is typically used to start a Django web page. If we properly installed and activated the environment, the result would be the development of the server at http://127.0.0.1:8000/. This constitutes the front end of the web page. For the execution of the back end, we execute a Python file (test.py) to test whether an accident occurred or not. This Python script performs accident detection in a video file using a pre-trained CNN model.

The following phases are the key workings of the accident detection model:

- Training phase, and
- Detection phase

In the training phase, we train the model using a Python script (train.py). Here, we are defining and training a convolutional neural network (CNN) for an image classification task using the Keras API from TensorFlow. The code is loading the training, validation, and testing datasets and configuring them for performance using TensorFlow's AUTOTUNE feature. The CNN model consists of convolutional layers, max-pooling layers, a flattening layer, and dense layers with ReLU and softmax activations. We are using the Adam optimizer and a sparse categorical cross-entropy loss function. The code also includes callbacks to save the best model weights during training and plot training and validation loss and accuracy. Finally, we are saving the model structure as a JSON file.

In the detection phase, a Python script (detection.py) defines a class called 'AccidentDetectionModel' that loads a pre-trained model from a JSON file and weights file and allows for prediction of whether an input image contains an accident or not. The class has a class variable called class_nums, which is a list containing the names of the two classes: "Accident" and "No Accident". The constructor __init__() takes as input the filenames of the model's JSON and weights files, loads the model from the JSON file using model_from_json(), and loads the weights into the model using load_weights(). The make_predict_function() method is called on the loaded model to ensure that it is thread-safe for use in multi-threaded environments. The predict_accident() method takes as input an image and returns a tuple containing the predicted class of the image and the model's prediction scores for each class, which are stored in the self.preds attribute of the class. The predicted class is obtained by finding the index of the maximum value in the prediction scores using np.argmax() and mapping it to the corresponding class name in AccidentDetectionModel.class_nums.

B. Traffic Violation Detection Model of TRA-BOT

This model of TRA-BOT is implemented as a software part and made real-time using laptop front camera. In traffic violations, we focus on over-speed tracking of the vehicles passing on the road that violate their speed above a threshold speed limit by using the ML algorithm [4]. If such a vehicle is detected by the model (a Haar cascade classifier of machine learning), the details of those vehicles will be recorded and saved in the police directory, and simultaneously, an alert SMS will be sent to the control room using the Twilio communication tool so that the police authorities can take the necessary actions against them so that we could prevent the occurrence of an accident due to over speeding.

The Haar Cascade Classifier is primarily used for object detection, but it can also be used to estimate the speed of vehicles. To do this, a series of images are captured using a camera placed at a fixed location, such as a traffic camera. The Haar Cascade Classifier is applied to each image to detect and track the vehicles. The speed of a vehicle can be estimated by comparing the positions of the vehicle in each frame. By knowing the time between each frame and the distance between the positions of the vehicle in each frame, the speed of the vehicle can be calculated. For example, if the camera captures 30 frames per second and the distance between the positions of the vehicle in two consecutive frames is 3 metres, then the vehicle is travelling at a speed of 90 metres per second, or approximately 324 kilometres per hour. It's worth noting that this method of speed estimation is not always accurate, as it assumes that the distance between the positions of the vehicle in two consecutive frames is solely due to the vehicle's movement. Factors such as camera jitter, road conditions, and other objects moving in the scene can introduce errors in the speed estimation. This classifier is implemented as an XML file (HaarCascadeClassifier.xml)

that defines the features and thresholds of a machine learning model for object detection, specifically for detecting vehicles and estimating their speed [10].

C. Traffic Density Detection Model of TRA-BOT

This model of TRA-BOT is implemented as a software part and made real-time using laptop front camera. In the traffic density part, we focus on predicting a normal or high-traffic road based on the provided dataset by using the classification model of DL [12]. If high traffic is predicted by the model (a binary classification model of deep learning), it passes an alert SMS to the control room (using the Twilio communication tool) that there is a high traffic density in the corresponding

GPS location (in the format lat/lon/ele) and there is a possibility of the occurrence of an accident. Thereby, the police authorities can take the necessary precautions to reduce traffic and prevent accidents. This algorithm classifies the dataset by calculating the step size of the given image. The step size is proportional to the traffic density [1].

This pre-trained binary classification model consists of several convolutional and pooling layers that learn hierarchical representations of the input image. The final layer is a dense layer with a sigmoid activation function that produces a single output between 0 and 1, representing the predicted probability of the input image belonging to the high traffic class. The loss function used is binary cross-entropy, and the optimizer used is Adam [10].

The script loads a trained Keras model from a file using the load_model() function. The loaded model is a custom CNN model for road traffic detection that was previously trained using a dataset of road traffic images. The model is loaded from the file "Model/Class/model_Class.h5" using the load_model() function from Keras.

IV. IMPLEMENTATION

The design in our system encompasses a wide range of activities, from creating rough sketches and wireframes of the system to outlining the user experience and user interface design. It can also involve designing the systems architecture, data models, and software and hardware specifications.

A. Specifications

The hardware and software requirements used here are:

i. Hardware requirements
- Processor: i5 or i7
- RAM: 8GB (Minimum)
- Hard Disk: 500GB or above
- Mobile Phone

ii. Software Requirements
- Tools: Python IDLE, Anaconda, Sublime Text, Twilio
- Python: version 3.6
- Operating System: Windows 10
- Front End : HTML, CSS, JavaScript
- Back End : Django (Python) Database: SQLite

Hardware specification is a critical component of hardware design, production, and procurement that is essential for ensuring that hardware devices meet the needs of their users. The following are the specifications used in TRA-BOT:

i. Hardware Part
- R-Pi camera interface with R-Pi using CSI camera port of the R-Pi
- Display device

ii. Embedded Devices
- Raspberry Pi 4 (4GB)
- 16GB Micro SD card for loading OS
- 5V 3A C type Charger for Raspberry Pi
- R-Pi Camera rev 1.3 for capturing video

B. Programming

The programming phase for both intelligence and real-time implementation is done in Python. The following table gives the details of the modules used in developing the programme.

The packages and libraries used for developing the Python code are listed in the table.

TABLE I
MODULES USED IN PYTHON

Sl No.	Modules	
	Package	**Library**
1.	Machine Learning: NumPy, Pandas, TensorFlow, Keras, PyTorch, Matplotlib, CV2	Utility Libraries: os, sys, dlib, PIL, shutil, picamera
2.	Web Development: Django	Date and Time: datetime
3.	GUI Development: Tkinter, Easygui	DB Management: sqlite3
4.	Communication: Serial, Twilio	Data Formats: json, xml, h5py

C. Connection Interface for Real-time

The Raspberry Pi board is connected to the interfacing components for making the code run in real-time. Here, in order to power the board, the 5V/3A C-type charger is connected to the USB-C power port. The OS of the Raspberry Pi is installed on the microSD card, which is to be inserted into the SD card slot.

The R-Pi camera is interfaced with the 2-lane MIPI CSI camera port of the board. A heat sink is attached, and a cooling fan is connected with the GPIO pins (Pins 4 and 6) of the board to maintain a lower temperature while running in real-time.

A Gigabit Ethernet port connects to the internet, and a monitor is connected to the board via the HDMI port to receive alerts if an accident occurs. Fig. 2 shows the schematic block representation of the connection interface.

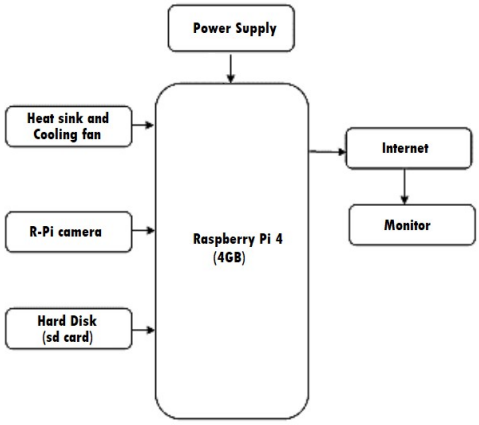

Fig.2 Block diagram of connection interface for real-time

V. EXPERIMENTATION

On detection of an accident, alerts are sent to both ends. The decision to be taken (either action or rejection) by the hospital or police departments is done through their corresponding login credentials. The front end and back end of the web page are developed by the 'manage.py runserver' and 'test.py' commands, respectively. Figs. 3 and 4 below show the progress of the development of our web page.

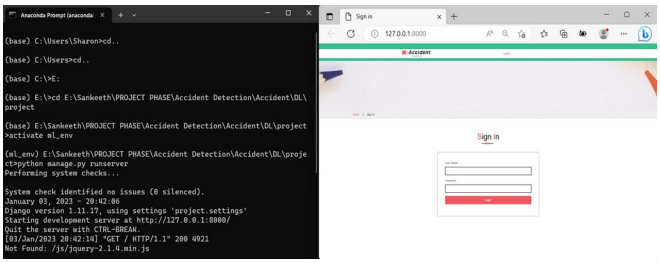

Fig.3 First console for front end of web page

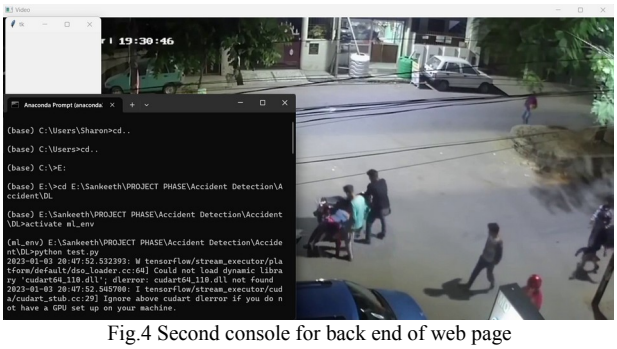

Fig.4 Second console for back end of web page

If an action is taken, it will be recorded on the web page; otherwise, it will be rejected and stored in an SQLite database. Figures 5, 6 and 7 show the results of the accident detection model of TRA-BOT.

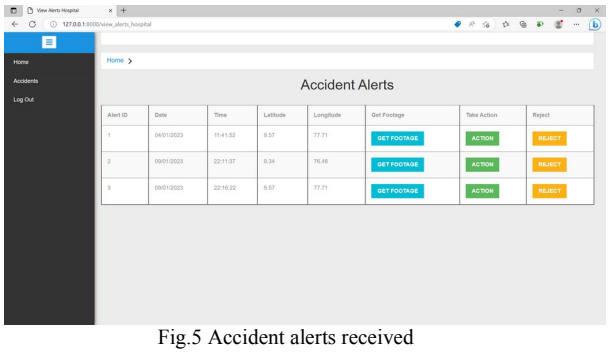

Fig.5 Accident alerts received

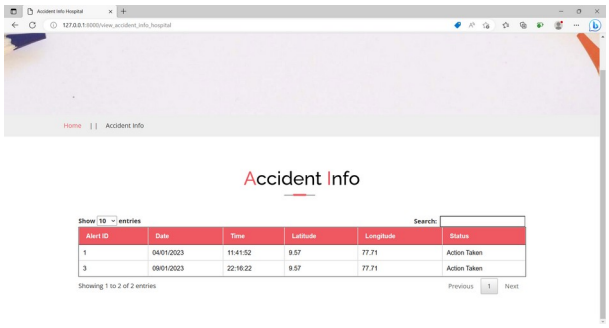

Fig.6 Table showing the list of action taken

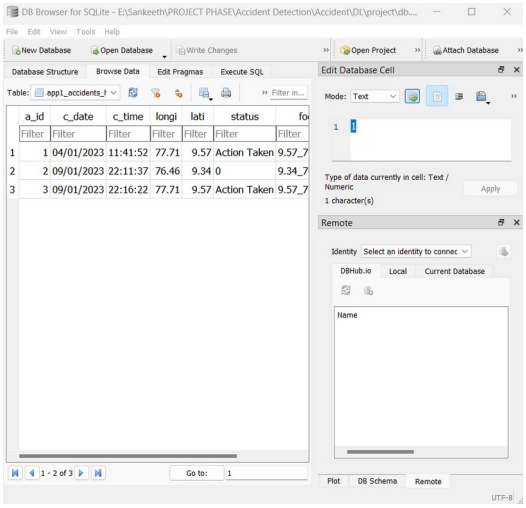

Fig.7 Database SQLite records

In the traffic violation part here, we consider only over speeding. If an over speeding vehicle is detected, the images of those vehicles will be cropped and saved in the police directory along with their details so that the authorities can take the necessary actions against them. Meanwhile, an alert SMS will also be sent to the police control room (mobile phone).

Fig. 8 shows the result of the speed detection system along with its console. Fig. 9 shows the recorded details of over speeding vehicles along with their details in the police directory.

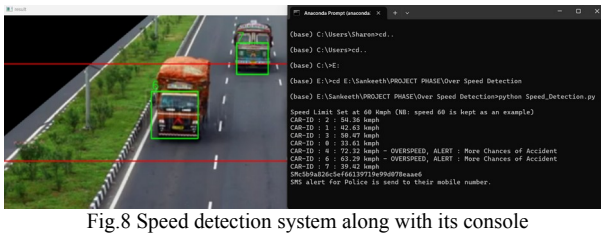

Fig.8 Speed detection system along with its console

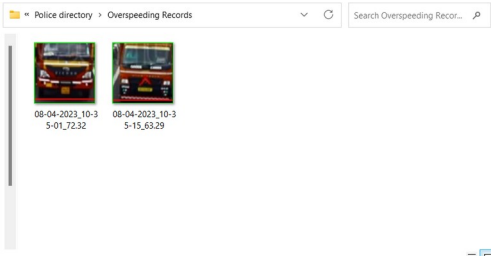

Fig.9 Records in the police directory for over speeding

In traffic density detection, we classify an image as normal or high traffic depending on the step size of the dataset image.

If normal traffic is predicted by the model, the class is neglected, and if high traffic is predicted, then it accounts for the occurrence of an accident and alerts the control room as an SMS to their mobile phone (indicating the GPS location of the traffic area) to reduce the traffic congestion.

Figures 10 and 11 show the results of traffic density along with its console. Fig. 12 shows the alert SMS sent to the police control room (both traffic violations and high traffic density) using the Twilio communication tool.

Fig.10 Normal traffic density

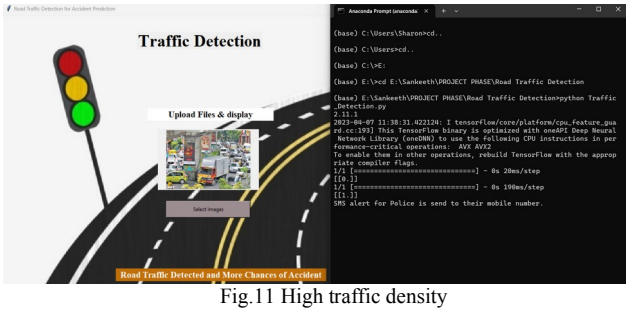

Fig.11 High traffic density

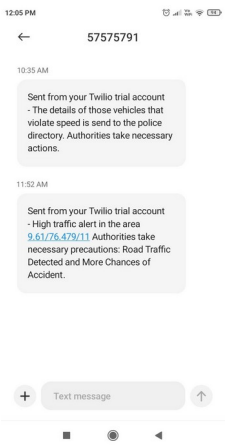

Fig.12 SMS alert given to police control room

Implementing the accident detection system of TRA-BOT as hardware using a Raspberry Pi board and R-Pi camera will require installing the necessary packages, modifying the software code, integrating the hardware and software, and testing the system to ensure that it is working correctly. Specifically, we need to modify the code to read video input from the Raspberry Pi camera instead of a file and to send the video footage to the hospital and police login credentials.

Overall, to modify the code for real-time processing, we will need to update the code that handles the video input, modify the settings and URL routes for the Django web page, and ensure that it is running on the correct IP address and port [5,11]. Fig. 13 shows the real-time interface of the R-Pi board and R-Pi camera connected to the system.

Fig.13 Real-time interface

VI. Conclusion

The designed model was successfully able to detect accidents in real time. Live footage was also sent to both the hospital and police departments. After verifying the credibility of the accident, they can accept or reject the alert. If the alert is accepted, services will be provided; otherwise, the alert can be rejected. This system helps in providing timely rescue to affected victims. These two authorities received the details of the accident through the web page that we developed. The decisions taken by them will be stored in an SQLite database.

This accident detection model of TRA-BOT uses a combination of image processing and deep learning. It uses a pre-trained CNN deep learning model (DetectNet) to predict whether an accident has occurred or not. The system also checks the possibility of an accident by considering traffic violations and traffic density detection.

In traffic violations, we focused on the over speeding of vehicles. Upon crossing the speed limit (that the user sets), the images of such vehicles were stored in the police directory along with their date, time and speed. Here, we used image processing (for image resizing) and a machine learning algorithm called the Haar Cascade classifier for vehicle detection and its speed estimation. In traffic density detection, a given image is classified as normal or high traffic based on the step size in ms. A lower step size indicates normal traffic density, and a higher step size indicates a higher traffic density. Here, we used image processing (for image resizing) and a CNN of deep learning called the binary classification model, which calculates the step size for traffic density detection. Meanwhile, a separate alert SMS is received by the police control room (here, our mobile phone) if a traffic violation or high traffic density is detected, assuming that both increase the possibility of an accident.

VII. FUTURE SCOPE

In this paper, we have focused an accident detection system using image processing and deep learning (CNN). The system detects accidents using a pre-trained CNN model and sends video footage of the accident to the hospital and police login credentials through a web page created using Django. We have also developed software for traffic violation detection, specifically over speeding and traffic density detection, with the use of these tools: Python programming, Anaconda prompt, Sublime Text, accident to the hospital and police login credentials through a web page created using Django. We have also developed software for traffic violation detection, specifically over speeding and traffic density detection that also proved to work in real-time, with the use of these tools: Python programming, Anaconda prompt, Sublime Text, SQLite, and Twilio.

For future scope, we propose expanding the scope of traffic violation detection beyond over speeding to include other types of traffic violations, such as running a red light or illegal parking. Additionally, we plan to improve the accuracy of our accident detection system by fine-tuning the pre-trained CNN model or exploring other deep learning techniques. We also suggest enhancing the hospital and police login web pages by adding more features, such as a dashboard for real-time updates on accidents and traffic violations.

Furthermore, we plan to develop a mobile application to provide users with real-time updates on accidents and traffic violations in their area.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our advisors, Dr. Riboy Cheriyan, Dr. Pradeep C., and Er. Nishanth P.R., for their invaluable guidance and support throughout the research process. We also wish to thank our institution for their financial support as well as the library's assistance in finding the necessary research materials. Finally, we are grateful to all of the research participants who generously gave their time and effort to this system.

REFERENCES

[1] Aiesha Roshan etal., "Traffic Density Control Using Deep Learning", *International Journal of Research in Engineering and Science (IJRES),* vol.10, no.6, pp.1780-1783, July 2022.

[2] Nicole Berx and Liliane Pintelon, "Examining the role of Safety in the Low Adoption Rate of Collaborative Robots", *9th CIRP Conference on Assembly Technology and Systems,* vol.106, no.1, pp.51-57, Apr. 2022.

[3] Dr. C. K. Gomathy etal., "Accident Detection and Alert System", *Journal of Engineering, Computing Architecture,* vol.12, no.3, pp.32-43, Mar. 2022.

[4] Roopa Ravish etal., "Intelligent Traffic Violation Detection", *2nd Global Conference for Advancement in Technology (GCAT),* pp.1-7, Oct. 2021.

[5] Nakul Lakhotia, "Video Streaming in Web Browsers with OpenCV Flask", *Towards Data Science,* Oct. 2020.

[6] Vittorio Astarita etal., "A Review of the use of traffic simulation for the evaluation of traffic safety levels: can we use simulation to predict crashes?", *23rd EURO Working Group on Transportation Meeting,* vol.52, no.4, pp.244-251, Sept. 2020.

[7] Faisal Ghaffar, "Controlling Traffic with Humanoid Social Robot", *ResearchGate* pp.1-10 Apr. 2020

[8] Mamatha T etal., "Smart Accident Detection And Alert Message System", *Global Journal of Research in Engineering Computer Sciences,* vol.2, no.4, pp.24-29, Feb. 2020.

[9] Ruben J Franklin and Mohana, "Traffic Signal Violation Detection using Artificial Intelligence and Deep Learning", *Fifth International Conference on Communication and Electronics Systems (ICCES),* pp.839-844, Jan. 2020.

[10] Usha Challa etal., "HawkAI: Monitoring Road Accidents Real-time Using Deep Learning", *MIDS Capstone Project,* Aug. 2019.

[11] S. Kumar and S. Arora, "Real-time Traffic Density and Overspeeding Detection System Using Deep Learning*", International Conference on Communication and Signal Processing,* vol.12, no.2, pp.207-217, Mar. 2019.

[12] Martin Kalus etal., "A Robotic Traffic Simulator for Teaching of advanced control methods", *ResearchGate, IFAC- Papers,* vol.49, no.6, pp.338-343, Dec. 2016.